

From Code to Mathematics: The Unified Particle Filter in `pfilter.jl`

Debsurya

September 8, 2026

Abstract

This document gives a line-by-line mathematical reading of the unified particle filter implemented in `src/pfilter.jl` of the `PartiallyObservedMarkovProcesses.jl` package. The filter generalizes the classical bootstrap particle filter along two axes: an effective-sample-size *trigger* that governs whether resampling occurs at a given observation time, and a tempering parameter *target* (β) that governs how much of a particle’s weight is discharged into selection probability versus retained on the surviving particle. We derive the algorithm from first principles and show that the two extreme settings recover the classical bootstrap filter and the fully weighted (never-resampling) filter respectively. Two genuine defects turned up in the course of this derivation and were corrected in the code: an earlier translation of the tempered resampling step indexed the retained weight by selection slot rather than by selected ancestor (Section 2), and the terminal ancestry draw used to report a simulated trajectory sampled uniformly over terminal particle indices rather than in proportion to their weight (Section 6). A third candidate defect was also derived, implemented, and then retracted: that a normalizing factor removed from the weights at resampling must be credited back into the running log likelihood. Section 3 shows this to be false — the uncredited bookkeeping actually used in the code (and, it turns out, in both of Aaron King’s reference implementations) is unbiased by a different route, the telescoping identity, and a paired numerical comparison against an exact Kalman likelihood for the Gompertz model shows no detectable difference between the credited and uncredited conventions. Section 7 gives the general validation against the exact Kalman likelihood, and Section 8 narrates the credit episode and what it does, and does not, show about the two genuine defects above.

Contents

1	The POMP model and the bootstrap particle filter	2
1.1	The partially observed Markov process	2
1.2	The bootstrap particle filter	3
2	Power-tempered systematic resampling	4
2.1	Tempering the resampling step	4
2.2	The four-argument <code>systematic_resample!</code>	4
3	The normalizing constant removed at resampling	6
3.1	The telescoping identity	6
3.2	Proper weighting: a sufficient condition	7
3.3	The removed factor, exactly	7
3.4	The telescoping route: unbiased without a credit	9

3.5	The numerical comparison	10
3.6	The code, and agreement with both of Aaron King’s implementations	10
4	ESS-triggered resampling	11
4.1	Effective sample size	11
4.2	<code>compute_ess_logLik!</code> : the two methods	11
4.3	The trigger	12
5	The filtering step	12
5.1	Classic path: <code>trigger = 1</code> , <code>target = 0</code>	13
5.2	General path: weighted, <code>trigger/target</code>	13
6	Weighted terminal ancestry draw	14
6.1	The problem	14
6.2	A two-particle counterexample	15
6.3	The corrected, five-argument <code>trace_ancestry!</code>	15
7	The Gompertz–Kalman validation	16
7.1	The log-Gompertz linear-Gaussian reduction	16
7.2	The scalar Kalman recursion	17
7.3	The Jacobian correction	17
7.4	Mapping the test	18
8	History: a credit added and retracted	19

1 The POMP model and the bootstrap particle filter

1.1 The partially observed Markov process

A partially observed Markov process (POMP) consists of a latent Markov process $\{X_t\}$ together with a sequence of noisy observations $\{Y_n\}_{n=1}^N$ made at times $t_1 < t_2 < \cdots < t_N$. Write $x_{0:N} = (x_0, x_1, \dots, x_N)$ for a realization of the latent process, sampled at $t_0 < t_1 < \cdots < t_N$, and $y_{1:N} = (y_1, \dots, y_N)$ for the data. The model is specified by three components:

- an *initial density*, from which X_0 is drawn: $X_0 \sim r_{\text{init}}(\cdot; \theta)$;
- a *process transition density* $f(x_n \mid x_{n-1}; \theta)$, giving the Markov transition law from t_{n-1} to t_n ;
- a *measurement density* $g(y_n \mid x_n; \theta)$, giving the conditional law of the observation at t_n given the latent state there, with observations conditionally independent given the latent path.

The likelihood of the data under parameter θ is the marginal density

$$\mathcal{L}(\theta) = \int \cdots \int r_{\text{init}}(x_0; \theta) \prod_{n=1}^N f(x_n \mid x_{n-1}; \theta) g(y_n \mid x_n; \theta) dx_0 dx_1 \cdots dx_N,$$

an integral of dimension proportional to N that is analytically intractable for essentially every model of interest beyond the linear-Gaussian case. In the code, `r_init`, the process model, and `g` are supplied as the basic components `rinit`, `rprocess`, and `logdmeasure`; the package requires only that one can *simulate* from `r_init` and from $f(\cdot \mid x_{n-1}; \theta)$, and *evaluate* $g(y_n \mid x_n; \theta)$ pointwise — the plug-and-play property that motivates the sequential Monte Carlo approach below.

1.2 The bootstrap particle filter

The bootstrap particle filter (Gordon, Salmond & Smith, 1993) estimates $\mathcal{L}(\theta)$ by propagating a swarm of J particles $\{X_n^{1:J}\}$ through the filtering recursion, using the process model itself as the proposal (importance) distribution. Initialize $X_0^j \sim r_{\text{init}}(\cdot; \theta)$, $j = 1, \dots, J$, independently. At each observation time $n = 1, \dots, N$:

1. **Prediction.** Propagate each particle forward through the process model: $X_n^j \sim f(\cdot \mid X_{n-1}^j; \theta)$, independently across j .
2. **Weighting.** Assign each predicted particle an importance weight equal to the measurement density evaluated at the data:
$$w_n^j = g(y_n \mid X_n^j; \theta).$$
3. **Resampling.** Draw J indices $\xi_n^1, \dots, \xi_n^J$ with $\Pr(\xi_n^k = j) \propto w_n^j$, and set the filtered particle $X_n^k \leftarrow X_n^{\xi_n^k}$ for each k , discarding the weights (each surviving particle now carries equal weight $1/J$).

Because the process model is used as the proposal, the importance weight is exactly the measurement density — no Radon-Nikodym correction between proposal and target transition kernels is needed. The conditional likelihood contributed at time n (the density of y_n given $y_{1:n-1}$) is estimated, unbiasedly, by the empirical mean of the weights:

$$\hat{\mathcal{L}}_n = \frac{1}{J} \sum_{j=1}^J w_n^j,$$

and, since $\mathcal{L}(\theta) = \prod_{n=1}^N \mathcal{L}_n(\theta)$ by the Markov property, the overall likelihood is estimated by the product

$$\hat{\mathcal{L}}(\theta) = \prod_{n=1}^N \hat{\mathcal{L}}_n = \prod_{n=1}^N \left(\frac{1}{J} \sum_{j=1}^J w_n^j \right).$$

This estimator is unbiased for $\mathcal{L}(\theta)$ (Del Moral, 2004) — a non-trivial fact, since it is a product of ratios of dependent random quantities, not simply an average of independent terms. The remainder of this document is concerned with a single question: which modifications of steps 1–3 above preserve this unbiasedness property, and what bookkeeping does the implementation require to make sure they do.

The package works throughout on the log scale, both for numerical stability (weights routinely span many orders of magnitude) and because the overall log likelihood decomposes as a sum,

$$\log \hat{\mathcal{L}}(\theta) = \sum_{n=1}^N \log \hat{\mathcal{L}}_n,$$

of the *conditional log likelihoods* — exactly the quantity accumulated, observation time by observation time, in the array `cond_logLik` in `pfilter.jl`.

2 Power-tempered systematic resampling

2.1 Tempering the resampling step

The classical resampling step (step 3 above) discharges the entire weight w^j of a particle into its selection probability. The unified filter instead factors each weight as

$$w = w^{1-\beta} \cdot w^\beta, \quad \beta \in [0, 1],$$

and uses only the first factor to drive selection: particle j is selected with probability proportional to $(w^j)^{1-\beta}$, and each selected particle *retains*, rather than discards, the second factor $(w^j)^\beta$ as its new weight — renormalized, as described below, to have unit mean over the particle swarm. The parameter β is called **target** in the code (and denoted β throughout this document to keep the two roles — the resampling parameter and the code identifier for it — visually distinct).

- At $\beta = 0$: selection probability $\propto w^1 = w$, retained weight $\propto w^0 = 1$. This is exactly the classical bootstrap resampling step: particles are chosen proportional to their full weight and emerge with equal weight.
- At $\beta = 1$: selection probability $\propto w^0 = 1$, retained weight $\propto w^1 = w$. Selection is now *uninformative* — every particle is equally likely to be chosen regardless of its weight — while the full weight is carried forward unchanged on the selected copy. No information is discarded; this is resampling in name only, and is used to reshuffle particle identity (for bookkeeping of ancestry) without touching the weighted representation.
- At intermediate β : a genuine interpolation. Some of the weight’s information is spent reducing the variance of the particle swarm’s spatial distribution (via selection), and the remainder is preserved as an informative weight on the surviving particles, to be combined multiplicatively with weights accrued at future observation times.

2.2 The four-argument `systematic_resample!`

The tempered resampling step is implemented in `src/pfilter.jl`, lines 450–482:

Listing 1: `systematic_resample!`, power-tempered systematic resampling (`pfilter.jl`, lines 450–482)

```
1 systematic_resample!(  
2     p::AbstractArray{I,1},  
3     w::AbstractArray{W,1},  
4     ucum::AbstractArray{W,1},  
5     beta::W,  
6 ) where {I,W} = begin  
7     @assert length(ucum)==length(w)==length(p)  
8     s::W = 0  
9     alpha = 1-beta  
10    @inbounds for j in eachindex(w)  
11        s += w[j]^alpha  
12        ucum[j] = s  
13    end  
14    n::I = length(w)  
15    i::I = 1  
16    du::W = s/n
```

```

17  u::W = -du*rand(W)
18  @inbounds for j in eachindex(p)
19      u += du
20      while (u > ucum[i] && i < n)
21          i += 1
22      end
23      p[j] = i
24  end
25  @inbounds for j in eachindex(p)
26      ucum[j] = w[p[j]]^beta
27  end
28  @inbounds for j in eachindex(w)
29      w[j] = ucum[j]
30  end
31  w ./= mean(w) # subsequent steps rely on the weights having unit mean
32  nothing
33 end

```

We map this line by line onto the mathematics of systematic resampling (Kitagawa, 1996), the standard low-variance alternative to multinomial resampling.

Cumulative sum of tempered weights (lines 459–462). Writing $w^{1:J}$ for the incoming (linear, not necessarily unit-mean) weight vector and $\alpha = 1 - \beta$, the code computes the cumulative sums

$$S'_j = \sum_{i=1}^j (w^i)^\alpha, \quad j = 1, \dots, J, \quad S' := S'_J = \sum_{i=1}^J (w^i)^\alpha.$$

`ucum[j]` holds S'_j ; `s` accumulates to S' at the end of the loop.

The systematic sweep (lines 463–473). A single uniform random offset $u_0 \sim \text{Unif}(0, S'/J)$ is drawn (line 466, as `-du*rand(W)`), so that the first increment lands at exactly this offset), and J equally spaced points

$$u_k = u_0 + k \cdot \frac{S'}{J}, \quad k = 1, \dots, J,$$

are swept against the cumulative sum $S'_{1:J}$: the ancestor index for selection k is

$$p_k = \min\{j : S'_j \geq u_k\}.$$

This is systematic resampling on the tempered weights $(w^j)^{1-\beta}$: each ancestor j is selected with multiplicity concentrated near $J(w^j)^{1-\beta}/S'$, and, unlike independent multinomial draws, the selection has minimal variance subject to these marginal selection probabilities — a single sorted sweep rather than J independent draws. `p[j]` on exit is the array $\xi^{1:J} = (p_1, \dots, p_J)$ of selected ancestor indices.

Ancestor-indexed retained weight (lines 474–476). This is the step at the mathematical heart of the tempered scheme, and the one most easily gotten wrong (Section 6 below describes an analogous error in ancestry tracing). Each original particle's weight factored as $w^j = (w^j)^{1-\beta}(w^j)^\beta$; the first factor has already been fully discharged into the selection step above (a particle with a large $(w^j)^{1-\beta}$ is selected many times). The remaining factor, $(w^j)^\beta$, is the piece of information

not yet used, and it belongs to the *particle*, not to the *selection slot* it lands in. Consequently the retained weight of the k -th selected particle must be indexed by its ancestor:

$$\tilde{w}_k = (w^{p_k})^\beta, \quad k = 1, \dots, J,$$

exactly what line 475 computes: `ucum[j] = w[p[j]]^beta`. (`ucum` is reused here purely as scratch space, unrelated to its earlier role as the cumulative-sum array.) The alternative — raising the weight vector to the power β *before* indexing by `p`, i.e. computing $w[j] \leftarrow w[j]^\beta$ in place and then permuting — assigns to selection slot k the quantity $(w^k)^\beta$, the retained-weight power of whichever particle originally occupied position k , which is in general unrelated to the particle that was actually selected there. This positional form was the form found in an earlier translation of the algorithm (see the memo in this directory, item 2); it targets the wrong measure whenever $\beta > 0$, as Section 6 demonstrates concretely by a two-particle example in the closely analogous setting of the terminal ancestry draw.

Writing back and renormalizing (lines 477–480). The retained weights computed in the preceding loop are written into `w` (lines 477–479), and `w` is then divided by its own realized mean $m = \frac{1}{J} \sum_k \tilde{w}_k$, computed and applied in the single line `w ./= mean(w)` (line 480), so that on exit `w` has unit mean over the J particles. This is an implementation convention, not a mathematical necessity: the routine could equally well leave the retained weights unnormalized. The choice to renormalize is made because *every other stage of the filter* — in particular `compute_ess_logLik!`, discussed in Section 4 — assumes an entering weight vector with unit mean, so that the log of its arithmetic mean, needed for the conditional log likelihood, can be read off directly. Keeping this invariant globally is what makes it possible for the filter to treat ”just resampled” and ”several steps since the last resampling” states identically at every subsequent step.

No return value. The renormalization divides every retained weight — and with it the swarm’s total mass — by the common factor m . One might expect a factor removed from the weights at this point to need crediting back into the running log likelihood, on the grounds that it carries likelihood-scale information. `systematic_resample!` does no such thing: the routine returns `nothing` (line 481), and the caller (Section 5) uses nothing from this call beyond the updated `p` and `w`. Section 3, next, works out exactly what becomes of the factor m removes, and why no credit is in fact owed.

3 The normalizing constant removed at resampling

3.1 The telescoping identity

Consider two consecutive observation times, with the filter having just computed, at time $n - 1$, a properly weighted particle representation of the filtering distribution: a swarm $\{(X_{n-1}^j, \omega_{n-1}^j)\}_{j=1}^J$ with $\mathbb{E} \left[\frac{1}{J} \sum_j \omega_{n-1}^j h(X_{n-1}^j) \right] = \mathbb{E}[h(X_{n-1}) \mid y_{1:n-1}]$ for every test function h , and normalized so $\frac{1}{J} \sum_j \omega_{n-1}^j = 1$ (unit mean). Propagating each particle through the process model and weighting by the measurement density at time n produces an updated properly weighted swarm for the joint smoothing distribution of X_n given $y_{1:n}$, with un-normalized weights $\omega_{n-1}^j \cdot g(y_n \mid X_n^j; \theta)$, and — because the incoming swarm already had unit mean and represents the filtering measure correctly — the conditional likelihood contribution is exactly the mean of these updated weights:

$$\mathcal{L}_n = \frac{1}{J} \sum_{j=1}^J \omega_{n-1}^j g(y_n \mid X_n^j; \theta).$$

This is the content of the telescoping (or "sequential") identity $\mathcal{L}(\theta) = \prod_n \mathcal{L}_n(\theta)$ specialized to a weighted particle representation: *as long as the entering weights have unit mean and properly represent the filtering measure, the conditional likelihood at each step is recovered simply by averaging the updated, un-normalized weights* — no correction term is needed for the propagate/weight half of the recursion. The delicate step is what happens next, at resampling, and — as this section works out at some length, having gotten it wrong once already — there turns out to be more than one way for that step to hand back a swarm for which the identity keeps working.

3.2 Proper weighting: a sufficient condition

After weighting, resampling replaces the properly weighted swarm $\{(X_n^j, w_n^j)\}$ — where, dropping the accumulated-weight notation ω used above, w_n^j denotes the current, possibly un-normalized weight at time n — with a new swarm $\{(X_n^{p_k}, \tilde{w}_n^k)\}_{k=1}^J$ obtained via power-tempered systematic resampling with parameter β . One sufficient condition for the telescoping identity to continue to hold undisturbed at the next step is that the new swarm remain *properly weighted* for the same filtering measure in the classical importance-sampling sense: for every test function h ,

$$\mathbb{E} \left[\frac{1}{J} \sum_{k=1}^J \tilde{w}_n^k h(X_n^{p_k}) \mid X_n^{1:J}, w_n^{1:J} \right] = \frac{1}{S} \sum_{j=1}^J w_n^j h(X_n^j), \quad S := \sum_{j=1}^J w_n^j,$$

i.e. the resampled swarm remains an unbiased (times a known constant) representation of the same weighted empirical measure. A direct calculation — using $\Pr(p_k = j) = (w_n^j)^{1-\beta}/S'$ with $S' = \sum_i (w_n^i)^{1-\beta}$, and retained weight $\tilde{w}_n^k = (w_n^{p_k})^\beta/m$ where m is whatever normalizing constant is applied — shows

$$\mathbb{E} \left[\tilde{w}_n^k h(X_n^{p_k}) \mid X_n^{1:J}, w_n^{1:J} \right] = \frac{1}{m} \sum_{j=1}^J \frac{(w_n^j)^{1-\beta}}{S'} (w_n^j)^\beta h(X_n^j) = \frac{1}{m S'} \sum_{j=1}^J w_n^j h(X_n^j).$$

Comparing with the target above, this sufficient condition holds exactly when $mS' = S$, i.e. when the retained weight of the k -th selected particle is taken to be

$$\tilde{w}_n^k = (w_n^{p_k})^\beta \cdot \frac{S'}{S},$$

which, when the incoming weights already have unit mean ($S = J$), is $(w_n^{p_k})^\beta \cdot S'/J$ — this is the correct, and the only, assignment that makes the resampled swarm properly weighted in the sense above. It is not, however, the assignment the code makes, and it is worth being precise about the gap between the two before asking whether that gap actually matters.

3.3 The removed factor, exactly

The code renormalizes to *unit mean* instead, dividing every retained weight $(w_n^{p_k})^\beta$ by the realized sample mean

$$m = \frac{1}{J} \sum_{k=1}^J (w_n^{p_k})^\beta$$

rather than by the value S/S' that proper weighting, above, would call for. Since m and S/S' are generically different numbers — m is a realized average over the J systematic selections, S/S' a

deterministic function of the pre-resampling weights alone — renormalizing to unit mean instead of to the properly-weighted target divides the swarm by an extra common factor

$$c = \frac{m}{S/S'} = \frac{m S'}{S} = \frac{m S'}{J}$$

(the last equality using the unit-mean convention $S = J$ maintained on entry to resampling, Section 4) relative to what a properly-weighted resampled swarm would carry. It was this factor c that an earlier draft of this document proposed crediting back into the running log likelihood — on the reasoning that, since c is exactly the deficit between the unit-mean convention and the properly-weighted sufficient condition of Section 3.2, restoring it would make the code satisfy that sufficient condition exactly, at every β . That reasoning is not wrong as far as it goes: crediting c back *would* restore proper weighting. What is wrong is the further step — taken, and since retracted — of concluding that failing to restore c therefore leaves the estimator biased. Failure of one sufficient condition is not evidence of bias; it is evidence only that a *different* argument is needed if the uncredited scheme is to be justified, and Section 3.4 supplies one.

The code computes c exactly, not merely in expectation, from the realized values of S' and m on each draw: writing $s = S' = \sum_j (w_n^j)^{1-\beta}$ (the running total accumulated in `ucum` over lines 459–462) and $m = \frac{1}{J} \sum_k (w_n^{p_k})^\beta$ (the realized mean computed inside `mean(w)` at line 480), $c = ms/n$ with n the particle count J . Proposition 1 below records exactly when this quantity is zero.

Proposition 1. *Suppose the incoming weight vector $w^{1:J}$ has unit mean, $\frac{1}{J} \sum_j w^j = 1$ (as guaranteed by `compute_ess_logLik!`, Section 4, on entry to resampling). Let $S' = \sum_j (w^j)^{1-\beta}$ and let m be the realized mean, over the J systematic selections $p_{1:J}$, of $(w^{p_k})^\beta$. Then $c = \log(mS'/J)$, the log of the factor by which unit-mean renormalization differs from the properly weighted target of Section 3.2, satisfies:*

- (a) $\log(mS'/J) = 0$ identically (for every realization of the resampling randomness) when $\beta = 0$;
- (b) $\log(mS'/J) = 0$ identically when $\beta = 1$;
- (c) for $0 < \beta < 1$, $\log(mS'/J)$ is generically nonzero, and its expectation over the resampling randomness is $\log\left(\frac{1}{J} \sum_j (w^j)^{1-\beta}\right) + \mathbb{E}[\log m]$, an $O(1/J)$ but generally nonzero quantity even when the pre-resampling weights $w^{1:J}$ are themselves exactly at their unit-mean expectation.

Proof. (a) At $\beta = 0$: $S' = \sum_j (w^j)^1 = \sum_j w^j = J$ (unit mean), and $m = \frac{1}{J} \sum_k (w^{p_k})^0 = \frac{1}{J} \sum_k 1 = 1$. So $mS'/J = 1 \cdot J/J = 1$ and $\log(mS'/J) = 0$, for every realization — this is simply the statement that classical resampling to equal weights removes no likelihood information: the entire mass $S = J$ (the unit-mean total) is preserved exactly, and no renormalization is needed ($m = 1$ trivially).

(b) At $\beta = 1$: $S' = \sum_j (w^j)^0 = J$, and $m = \frac{1}{J} \sum_k (w^{p_k})^1 = \frac{1}{J} \sum_k w^{p_k}$. Since selection at $\beta = 1$ is uniform systematic resampling (every $(w^j)^0 = 1$, so all ancestors are equally likely), and systematic resampling from a uniform base measure over J categories, each with population fraction exactly $1/J$ represented once, reproduces each index exactly once (systematic resampling against a perfectly uniform cumulative distribution is a bijection, not merely unbiased): p is a permutation of $1, \dots, J$, hence $m = \frac{1}{J} \sum_k w^{p_k} = \frac{1}{J} \sum_j w^j = 1$ (unit mean, again). So $mS'/J = 1 \cdot J/J = 1$ and $\log(mS'/J) = 0$, for every realization: at $\beta = 1$ resampling is a relabeling that touches neither the weights nor their sum.

(c) For $0 < \beta < 1$, neither of the two special structures above holds: $S' = \sum_j (w^j)^{1-\beta}$ is, by strict concavity of $x \mapsto x^{1-\beta}$ on $x > 0$ together with Jensen’s inequality, generically different from

J (equal to J only in the degenerate case of exactly equal weights, by strict concavity), and the selection multiplicities are no longer exactly uniform, so $m \neq 1$ in general. Hence $mS'/J \neq 1$ generically and $\log(mS'/J) \neq 0$. Taking expectations over the resampling randomness (which enters only through m , since S' is a deterministic function of the entering weights) gives the stated expectation formula. $\square$

Corollary 1. *The factor $c = \log(mS'/J)$ of Proposition 1 is identically zero whenever the filter is run at its classical default ($\beta = \text{target} = 0$) or at the fully weighted extreme ($\beta = 1$), and is generically nonzero at every intermediate β . At those two extremes, then, crediting c back or not makes no difference of any kind, to any realization: the credited and uncredited estimators coincide exactly, term by term, not merely in expectation. Only at intermediate β do the two conventions diverge at all, and it is precisely there that the question of whether the uncredited convention is still unbiased has to be settled by an argument that does not depend on proper weighting.*

3.4 The telescoping route: unbiased without a credit

The code, as it stands, takes neither credited path: `systematic_resample!` returns `nothing`, and `c` is never added anywhere. This is precisely the bookkeeping already used, for entirely unrelated reasons of implementation convenience, in the R `pomp` package’s particle filter (`wfilter.c`): weights are carried forward without any resampling credit, and each observation time’s conditional log likelihood is instead computed as the log of a *ratio* of successive total masses, `loglik = maxlogw + log(w) - log(olddw)` (`wfilter.c`, line 110), where `olddw` is simply whatever total mass the carried weights happen to have on entry to that step — resampled and renormalized, or not. An earlier note on this point claimed that the R implementation “accounts for” the mass removed at resampling “by construction.” That claim was also wrong, in the same direction as the credit proposal it was meant to support: R credits nothing at resampling either. The mass change across a resampling step is simply never entered into the likelihood at all; the next observation’s increment is taken relative to whatever baseline the carried weights land on, with no reference to what that baseline “should” have been. Aaron King’s R and Julia implementations compute, and always have computed, the *same* estimator.

Why does this not introduce bias? Return to the telescoping identity of Section 3 above: $\mathcal{L}_n$ is recovered as the mean of the carried weight times the next measurement density, whatever the overall scale of the carried weight happens to be, *provided* that scale is tracked consistently — i.e. provided the very same carried weights, at whatever normalization resampling left them, are the ones used, unmodified, in the following step’s mean. This is exactly what the code does: the factor c that unit-mean renormalization removes at resampling is not credited to the likelihood at that step, but neither is it discarded outright — it simply becomes part of the baseline that the *next* step’s ratio is computed against. Where the properly-weighted route of Section 3.2 restores the identity by crediting c at the moment it is removed, the telescoping route restores it by letting c ride, silently, into the denominator of the next observation’s likelihood ratio. These are two different bookkeeping conventions for the same recursion, not a correct one and an incorrect one — provided the silent carry-forward is itself unbiased, which the following calculation makes plausible: c is a conditionally mean-one random factor,

$$\mathbb{E}[m] = \sum_j \pi_j (w^j)^\beta = \sum_j \frac{(w^j)^{1-\beta}}{S'} (w^j)^\beta = \frac{1}{S'} \sum_j w^j = \frac{J}{S'} \quad (\text{using } \sum_j w^j = J),$$

so $\mathbb{E}[c] = \mathbb{E}[mS'/J] = (J/S')(S'/J) = 1$ exactly, for every β and every realization of the pre-resampling weights. A factor with conditional mean exactly one, absorbed silently into the next

step’s baseline, changes no expectation; it can only add variance. Section 3.5 checks this directly rather than resting on the plausibility argument alone.

3.5 The numerical comparison

Because the two conventions differ only in a single scalar bookkeeping choice — whether c is multiplied into the likelihood at the step it is computed, or left to ride silently into the next step’s baseline — they can be compared *exactly*, from the very same particle draws: run the tempered filter once, form both the credited and the uncredited running log likelihood from the identical sequence of particle positions and weights, and compare. Because the comparison is paired in this way, its standard error reflects only the (tiny) scalar difference between the two accumulators, not the much larger Monte Carlo variability of the likelihood estimate itself.

This comparison was run on the linear-Gaussian reduction of the Gompertz model used for the exact-likelihood validation of Section 7, at every combination of $\beta \in \{0.25, 0.5, 0.75\}$ and $J \in \{8, 16, 64\}$, with 400,000 paired replicates at each combination (particle counts kept deliberately small, since Proposition 1(c) identifies c ’s expected deviation from one as $O(1/J)$, and the effect, if real, should be easiest to see there). Representative rows, the difference of the two estimators’ means with its paired standard error and z -score:

J	β	$\mathbb{E}[\widehat{\mathcal{L}}_{\text{credited}}] - \mathbb{E}[\widehat{\mathcal{L}}_{\text{uncredited}}]$	SE	z
8	0.50	−0.000914	0.000451	−2.02
16	0.50	−0.000175	0.000196	−0.89
64	0.50	−0.000062	0.000061	−1.02

Across all nine combinations tried, the two estimators’ expectations agree to within 0.003% of $\widehat{\mathcal{L}}$, with no statistically significant difference at any (β, J) . This is exactly what $\mathbb{E}[c] = 1$ predicts, and exactly what it does not, by itself, guarantee: mean one is consistent with the two schemes differing in variance while agreeing in expectation, which is what is observed. At $J = 8$, $\beta = 0.5$, c itself (the total per-step credit, in log) has mean -0.0050 and standard deviation 0.0998 — a relative spread of roughly ten percent — shrinking to mean -0.0005 , standard deviation 0.0319 at $J = 64$. Crediting c back therefore does not correct a bias, because there is none to correct; it substitutes a small, mean-one multiplicative correction, with its own sampling variance, for a status quo that already had none.

3.6 The code, and agreement with both of Aaron King’s implementations

The code accordingly credits nothing at resampling. The retained weights are renormalized to unit mean, as Section 2 already described, and the routine returns `nothing`:

Listing 2: The end of `systematic_resample!` (`pfilter.jl`, lines 477–482): no credit computed, no credit returned

```

1  @inbounds for j in eachindex(w)
2      w[j] = ucum[j]
3  end
4  w ./= mean(w) # subsequent steps rely on the weights having unit mean
5  nothing
6  end

```

Listing 3: The caller, `pfilt_step_comps!` (`pfilter.jl`, line 315): the return value is not used

```
1 systematic_resample!(p, w, work, target)
```

With this bookkeeping, `cond_logLik[k]` — computed entirely by `compute_ess_logLik!` from the propagate/weight step, and left untouched by resampling — is, at every observation time n and for every $\beta \in [0, 1]$ and every resampling schedule, an unbiased contribution to the overall telescoping product by the route of Section 3.4 rather than the route of Section 3.2, restoring the identity $\hat{\mathcal{L}}(\theta) = \prod_n \hat{\mathcal{L}}_n$ as an unbiased estimator of $\mathcal{L}(\theta)$ uniformly over the trigger/target parameter space, without any resampling-time correction term. This is also, as Section 3.4 noted, precisely the convention R’s `pomp` has used all along: this implementation and both of Aaron King’s now agree, and always did, on this point. Section 7 validates the filter as a whole numerically against an exact reference likelihood; Section 3.5 above is the numerical comparison specific to this bookkeeping question.

4 ESS-triggered resampling

4.1 Effective sample size

The effective sample size of a weighted particle swarm $w^{1:J}$ is the classical diagnostic

$$\text{ESS}(w^{1:J}) = \frac{\left(\sum_{j=1}^J w^j\right)^2}{\sum_{j=1}^J (w^j)^2},$$

a quantity ranging from 1 (all mass on one particle — maximal weight degeneracy) to J (all weights equal — no degeneracy), invariant to a uniform rescaling of the weights. It is the standard proxy for “how many independent, equally weighted particles is this weighted swarm worth,” and is the basis for adaptive resampling schedules: resample only when the swarm has become sufficiently degenerate to threaten the accuracy of the filtering approximation, rather than at every step regardless of need.

4.2 `compute_ess_logLik!`: the two methods

Two methods of `compute_ess_logLik!` appear in the code (lines 330–403), corresponding to the classic and general engines respectively.

Classic method (lines 330–360), no carried weight. Given only the incremental log weights $\log w_n^j = \log g(y_n \mid X_n^j; \theta)$ (no prior weight carried in, since the classic engine always resamples to equal weights at the previous step), the routine:

- finds $\ell_{\max} = \max_j \log w_n^j$ (line 335) and subtracts it from every log weight (line 344), the standard log-sum-exp stabilization, so that the largest stabilized weight is 1 before exponentiating;
- accumulates $s = \sum_j v^j$ and $ss = \sum_j (v^j)^2$ with $v^j = \exp(\log w_n^j - \ell_{\max})$ (lines 341–348);
- sets $\text{ESS} = s^2/ss$ (line 350), exactly the definition above, computed on the stabilized (equivalently, unstabilized — ESS is scale-invariant) weights;
- computes the conditional log likelihood as $\log \hat{\mathcal{L}}_n = \ell_{\max} + \log(s/J)$ (lines 349–352), the log-sum-exp identity $\log\left(\frac{1}{J} \sum_j w_n^j\right) = \ell_{\max} + \log\left(\frac{1}{J} \sum_j e^{\log w_n^j - \ell_{\max}}\right)$;

- overwrites `logw` with the log weights renormalized to have mean-one linear scale, $\log w_n^j - \log(s/J)$ (line 353), so that *even along the path where resampling does not subsequently occur*, the weights handed onward are on a canonical, likelihood-consistent scale (used, for instance, directly by `systematic_resample!`'s classic sibling method, or, in the general engine, becoming the next step's carried weight).

If $\ell_{\max}$ is not finite (line 340's `else` branch), every particle has zero measurement density — the swarm has been exhausted, filter failure — and the routine reports $\text{ESS} = 0$, $\log \hat{\mathcal{L}}_n = -\infty$ (lines 355–357).

General method (lines 368–403), folding in a carried weight. The general engine carries a linear, unit-mean weight vector `w` forward across observation times whenever resampling does not occur at every step (Section 5); this method folds that carried weight into the current step's incremental log weight before computing ESS and the conditional log likelihood, and — critically — writes the folded, renormalized result *back into* `w` on exit (line 395, `w ./= lik`), maintaining the unit-mean invariant on which every other routine in the filter relies. Concretely:

$$\log \tilde{w}^j = \log w_{\text{carry}}^j + \log w_n^j - \ell_{\max} \quad (\text{line 384}),$$

combining the incoming (unit-mean) carried weight multiplicatively with the new incremental weight, again with the log-sum-exp stabilization $\ell_{\max} = \max_j \log w_n^j$ (the incremental weight's own maximum, not the combined one — since the carried weight already has unit mean and no extreme values by construction). ESS and $\log \hat{\mathcal{L}}_n$ are computed exactly as before from $s = \sum_j \exp(\log \tilde{w}^j)$ and $ss = \sum_j \exp(\log \tilde{w}^j)^2$, with the comment on line 390 flagging explicitly that the formula $\log \hat{\mathcal{L}}_n = \ell_{\max} + \log(s/J)$ is valid here *only because* the carried weight entered with unit mean — the same telescoping-identity fact derived in Section 3. On exit, `w` is renormalized to unit mean (line 395) and `logw` is renormalized to zero mean on the log-linear scale (line 394), so that whichever code path runs next (resampling or not) sees a canonically scaled representation.

4.3 The trigger

The trigger threshold, `trigger` $\in [0, 1]$ (denoted here by the same identifier, since it plays no separate Greek-letter role in the mathematics), governs whether resampling occurs at all: resampling is invoked whenever

$$\text{ESS}_n \leq \text{trigger} \cdot J,$$

tested at line 314 (`ess[] ≤ trigger*n`), *and* the maximum log weight is finite (a degenerate/exhausted swarm always triggers the fallback path, never the resampling path, since there is nothing informative to resample). `trigger = 1` resamples unconditionally, since $\text{ESS}_n \leq J$ always; `trigger = 0` never resamples via this branch (short of exact degeneracy), since $\text{ESS}_n \geq 1 > 0$ always for a non-degenerate swarm.

5 The filtering step

`pfilt_step_comps!` assembles one full observation-time update: weighting statistics, the resampling decision, and (when triggered) the resampling operation itself, dispatched to one of two methods according to the number of arguments supplied — a compile-time dispatch on whether the classic or general engine is in use, resolved once, outside the per-step loop, by the choice made in `pfilter` at lines 98–136 of which `pfilter.internal!` method to call.

5.1 Classic path: trigger = 1, target = 0

Listing 4: Classic filtering step (pfilter.jl, lines 271–293)

```

1 pfilt_step_comps!(
2   logLik::AbstractArray{W,0},
3   ess::AbstractArray{W,0},
4   logw::AbstractArray{W,1},
5   p::AbstractArray{I,1},
6   xp::AbstractArray{X,1},
7   xf::AbstractArray{X,1},
8   work::AbstractArray{W,1},
9   resamp::AbstractArray{Bool,0},
10  n::Integer = length(logw),
11 ) where {W<:AbstractFloat,I<:Integer,X<:NamedTuple} = begin
12   logwmax = compute_ess_logLik!(ess, logLik, logw)
13   if isfinite(logwmax)
14     systematic_resample!(p, logw, work)
15     @inbounds xf .= xp[p]
16     resamp[] = true
17   else
18     p .= collect(eachindex(p))
19     xf .= xp
20     resamp[] = false
21   end
22   nothing
23 end

```

This is the algorithm of Section 1.2 exactly: $\log \hat{\mathcal{L}}_n$ and ESS_n are computed from the incremental weights alone (no carried weight — every step starts from equally weighted particles, since the classic engine resamples unconditionally whenever the swarm is non-degenerate), then classical (untempered, $\beta = 0$) systematic resampling is applied whenever the weights are informative ($\log w_{\max}$ finite), replacing X_n^p for the filtered particles X_n^f (line 285, `xf .= xp[p]`). In the degenerate case, the identity permutation is used and the predicted particles are carried through unchanged as the filtered particles (lines 288–290) — the swarm has failed, but the routine does not error, allowing the caller to detect and report filter failure through $\log \hat{\mathcal{L}}_n = -\infty$ downstream.

5.2 General path: weighted, trigger/target

Listing 5: General filtering step (pfilter.jl, lines 299–324)

```

1 pfilt_step_comps!(
2   logLik::AbstractArray{W,0},
3   ess::AbstractArray{W,0},
4   logw::AbstractArray{W,1},
5   p::AbstractArray{I,1},
6   xp::AbstractArray{X,1},
7   xf::AbstractArray{X,1},
8   w::AbstractArray{W,1},
9   work::AbstractArray{W,1},
10  trigger::W,
11  target::W,
12  resamp::AbstractArray{Bool,0},

```

```

13   n::Integer = length(logw),
14 ) where {W<:AbstractFloat,I<:Integer,X<:NamedTuple} = begin
15   logwmax = compute_ess_logLik!(ess, logLik, logw, w)
16   if isfinite(logwmax) && ess[] <= trigger*n
17     systematic_resample!(p, w, work, target)
18     @inbounds xf .= xp[p]
19     resamp[] = true
20   else
21     p .= collect(eachindex(p))
22     xf .= xp
23     resamp[] = false
24   end
25   nothing
26 end

```

The general step folds the carried weight w into the incremental weight via `compute_ess_logLik!`'s second method (Section 4), then resamples only when both the swarm is non-degenerate *and* the trigger condition $ESS_n \leq \text{trigger} \cdot J$ is met — otherwise the entire weighted particle representation is carried forward untouched (lines 319–321), with `logLik[]` left at the value set by `compute_ess_logLik!` alone. When resampling does occur, it is tempered resampling at the supplied $\beta = \text{target}$ (line 315); as Section 3 works out in detail, `logLik[]` is left untouched by this call as well — the factor removed by the resampling routine's own unit-mean renormalization is not credited here, but is instead retired, unbiasedly, as part of the baseline the *next* observation's likelihood ratio is computed against.

Comparing the two methods: setting `trigger = 1` and `target = 0` in the general path reduces it, step for step, to the classic path — $ESS_n \leq 1 \cdot J$ always holds, so resampling always triggers, and $\beta = 0$ tempered resampling coincides with classical resampling exactly, the factor mS'/J of Proposition 1(a) being identically one at $\beta = 0$, so no discrepancy of any kind arises between the two paths at this setting. The classic method is retained separately in the code as a specialized, slightly cheaper implementation of this special case (no carried-weight bookkeeping, no trigger test to evaluate), not because the mathematics differs.

6 Weighted terminal ancestry draw

6.1 The problem

At the end of a filtering run, the package reports a single simulated trajectory $X_{0:N}^*$ from (an approximation to) the smoothing distribution, obtained by tracing the ancestry of one randomly selected terminal particle back through the recorded resampling permutations `perm`. When the terminal particle swarm is *equally* weighted — guaranteed under the classic engine, where every step resamples to equal weights — a uniform draw over the J terminal particles is exactly correct: each terminal particle already represents an equal share of the filtering measure's mass. But whenever `target > 0` or `trigger < 1`, the terminal swarm can carry unequal weights $w^{1:J}$ (accumulated exactly as in Sections 2–4), and a uniform draw over particle *indices* no longer targets the filtering measure: it implicitly reweights every particle to $1/J$, discarding the very information the weights encode.

6.2 A two-particle counterexample

Example 1. Take $J = 2$ terminal particles with weights $w^1 = 3, w^2 = 1$ (unit mean: $\frac{1}{2}(3+1) = 2 \neq 1$ in this raw form, but the argument is unaffected by an overall rescaling — take $w^1 = 1.5, w^2 = 0.5$ for the unit-mean version if preferred). The properly weighted representation of the filtering distribution at this final time is

$$\pi(\cdot) \propto w^1 \delta_{X^1}(\cdot) + w^2 \delta_{X^2}(\cdot) = \frac{3}{4} \delta_{X^1}(\cdot) + \frac{1}{4} \delta_{X^2}(\cdot)$$

after normalizing: particle 1 should be selected as the terminus of the reported trajectory with probability $3/4$, particle 2 with probability $1/4$. A uniform draw, $j \sim \text{Unif}\{1, 2\}$, instead selects each particle with probability $1/2$ — targeting the unweighted empirical measure $\frac{1}{2}\delta_{X^1} + \frac{1}{2}\delta_{X^2}$, a different distribution whenever $w^1 \neq w^2$, and one with no claim to approximating the filtering distribution of interest. The discrepancy is not a higher-order or asymptotically vanishing effect: it persists at every J , since it is a first-moment (bias in the selection probability), not a sampling-variance, error.

This is precisely the defect flagged, as a marginal question rather than a firm claim, against the uniform draw in an earlier version of `trace_ancestry!` (memo, item 4): "I noticed your marginal note there, questioning whether the first index in the ancestry is right — I suspect this is exactly the issue."

6.3 The corrected, five-argument trace_ancestry!

Listing 6: Weighted ancestral-lineage tracing (`pfilter.jl`, lines 508–544)

```

1 trace_ancestry!(
2     traj::AbstractArray{X,1},
3     filt::AbstractArray{X,2},
4     perm::AbstractArray{I,2},
5     w::AbstractArray{W,1},
6     work::AbstractArray{W,1},
7 ) where {X,I<:Integer,W<:AbstractFloat} = begin
8     @assert size(traj,1)==size(perm,1)
9     @assert size(filt)==size(perm)
10    @assert length(w)==size(perm,2)
11    n = length(w)
12    wmax::W = -Inf
13    for k in eachindex(w)
14        @inbounds wmax = (w[k] > wmax) ? w[k] : wmax
15    end
16    j::I = if isfinite(wmax)
17        s::W = 0
18        for k in eachindex(w)
19            @inbounds v::W = w[k]
20            s += v
21            @inbounds work[k] = s
22        end
23        u::W = s*rand(LogLik)
24        jj::I = 1
25        while (u > work[jj] && jj < n)
26            jj += 1
27        end

```

```

28     jj
29     else
30         rand(axes(perm,2))
31     end
32     for i in Iterators.reverse(axes(perm,1))
33         @inbounds traj[i] = filt[i,j]
34         @inbounds j = perm[i,j]
35     end
36     j
37 end

```

Cumulative weight construction (lines 523–529). Given the terminal (unit-mean, linear) weight vector $w^{1:J}$, the routine forms the cumulative sums $W_k = \sum_{i=1}^k w^i$, $k = 1, \dots, J$ — `work[k]` — exactly as `systematic_resample!` does for the tempered selection weights, but here for a *single* draw rather than a systematic sweep of J draws.

Weighted single draw (lines 530–535). A draw $u \sim \text{Unif}(0, W_J)$ is made (line 530, `s*rand(LogLik)`), with $W_J = s = \sum_k w^k$, and the smallest index j with $W_j \geq u$ is found by linear search — an inverse-CDF draw from the discrete distribution with $\Pr(J = k) = w^k / \sum_i w^i$, exactly the weighted selection probability required by the properly weighted representation. This is the correction to the counterexample above: with $w^1 = 3, w^2 = 1$, $u \sim \text{Unif}(0, 4)$, and particle 1 is selected whenever $u \in (0, 3]$ — probability $3/4$, as required.

Degenerate fallback (line 537). If every weight is $-\infty$ on the log scale (here checked via a $-\infty$ sentinel on the linear scale, `wmax` not finite — mirroring the fallback convention used throughout `pfilter.jl`), the routine falls back to a uniform draw, matching the equal-weight case and the behavior of the unweighted, four-argument `trace_ancestry!` (lines 483–496) used by the classic engine, where every terminal particle carries equal weight by construction and this method is unnecessary (and unused).

Backward trace (lines 539–543), identical in both methods. Once the terminal index j is fixed (by whichever draw was appropriate), the trajectory is traced backward through the recorded ancestor permutations exactly as in the unweighted case: at each observation time i , from the last to the first, `traj[i] ← filt[i, j]`, then `j ← perm[i, j]`, walking the lineage of ancestors back to time t_0 . This part of the routine is unaffected by weighting — the weighting question is entirely about *which* terminal lineage to initiate, not how to trace one once chosen — and is shared, verbatim in structure, between the two methods.

7 The Gompertz–Kalman validation

7.1 The log-Gompertz linear-Gaussian reduction

The Gompertz population model, in the parametrization used by the package’s built-in example, is

$$X_t = X_{t-1}^S K^{1-S} \varepsilon_t, \quad S = e^{-r}, \quad \varepsilon_t \sim \text{LogNormal}(0, \sigma_p^2),$$

with a lognormal measurement model, $\text{pop}_t \mid X_t \sim \text{LogNormal}(\log X_t, \sigma_m^2)$. Writing $Z_t = \log X_t$, taking logs of the process equation gives an exactly linear, Gaussian autoregression:

$$Z_t = (1 - S) \log K + S Z_{t-1} + w_t, \quad w_t \sim \text{Normal}(0, \sigma_p^2),$$

and the measurement model becomes, on the log scale, another linear-Gaussian observation equation:

$$\log(\text{pop}_t) = Z_t + v_t, \quad v_t \sim \text{Normal}(0, \sigma_m^2).$$

This is a scalar linear-Gaussian state-space model, for which the likelihood of $\log(\text{pop}_{1:N})$ is available exactly, with no Monte Carlo error, via the Kalman filter — a rare and valuable property for validating a Monte Carlo filter implementation, since the comparison is against a deterministic reference rather than a second noisy estimate.

In the package’s built-in Gompertz example, `rinit` sets X_0 to the parameter `X0` deterministically (no process noise at initialization), and the first observation time coincides with t_0 , so no process step intervenes before the first observation is assimilated: the filtering distribution of Z at the first observation time is a point mass, $Z_1 = \log(X_0)$ with zero variance. Every subsequent observation time is preceded by exactly one Gompertz recursion at unit time step, matching the annual spacing of the built-in data; the `discrete_time` step function used by the model ignores its `dt` argument and applies σ_p directly as the per-step log-scale standard deviation, so no $\sqrt{dt}$ rescaling is needed to match the recursion above exactly at $dt = 1$.

7.2 The scalar Kalman recursion

For $t = 1, \dots, N$, with filtering mean and variance m_{t-1}, v_{t-1} of Z_{t-1} given $\log(\text{pop}_{1:t-1})$ (and $m_1 = \log(X_0)$, $v_1 = 0$ at $t = 1$, reflecting the deterministic initialization above):

Predict (skipped at $t = 1$): $m \leftarrow (1 - S) \log K + S m$, $v \leftarrow S^2 v + \sigma_p^2$.

Innovate: forecast variance $F = v + \sigma_m^2$; innovation $e = \log(\text{pop}_t) - m$; contribution to the log likelihood $-\frac{1}{2} (\log(2\pi F) + e^2/F)$.

Update: Kalman gain $\kappa = v/F$; $m \leftarrow m + \kappa e$; $v \leftarrow v - \kappa^2 F$.

Summing the innovation contributions over $t = 1, \dots, N$ gives $\log \mathcal{L}_{\log y}$, the exact conditional log likelihood of $\log(\text{pop}_{1:N})$ under the linear-Gaussian reduction — precisely the computation performed by `kalman_loglik_logy` in `test/gompertz_kalman.jl`.

7.3 The Jacobian correction

The package’s `logdmeasure` for the Gompertz model evaluates the LogNormal density of pop_t itself, not of $\log(\text{pop}_t)$. Since $\text{pop}_t = \exp(\log(\text{pop}_t))$, the two densities are related by the change-of-variables Jacobian $f_{\text{pop}}(y) = f_{\log \text{pop}}(\log y)/y$, so, summing the log Jacobian factor $-\log y_t$ over all observations,

$$\log \mathcal{L}(\text{pop}_{1:N}) = \log \mathcal{L}(\log(\text{pop}_{1:N})) - \sum_{t=1}^N \log(\text{pop}_t).$$

The particle filter, run with the package’s actual `logdmeasure` (a density on the natural, untransformed population scale, as it must be to match the data), estimates $\log \mathcal{L}(\text{pop}_{1:N})$ — the left-hand side — while the Kalman recursion above computes $\log \mathcal{L}_{\log y} = \log \mathcal{L}(\log(\text{pop}_{1:N}))$, the first term on the right. The two must therefore differ by exactly the Jacobian sum, a fixed constant (independent of θ -estimation Monte Carlo error, and indeed independent of the particle filter’s behavior altogether) equal to `jacobian` = $-\sum_t \log y_t$ in the test code.

7.4 Mapping the test

`test/gompertz_kalman.jl` implements exactly this comparison.

Listing 7: Two checks against the exact Kalman likelihood (`test/gompertz_kalman.jl`, lines 89–117, abridged)

```

1 Random.seed!(20260907)
2 Nps = [200,1000,5000]
3 nreps = 30
4 lme = [
5     logmeanexp([logLik(pfilter(P,Np=Np,params=p1)) for _ in 1:nreps],se=true)
6     for Np in Nps
7 ]
8 gap = [x.est - ll_logy for x in lme]
9 se = [x.se for x in lme]
10
11 ## primary check
12 tol_spread = 6*maximum(se)
13 @test maximum(gap)-minimum(gap) < tol_spread
14
15 ## secondary check
16 for (g,s) in zip(gap,se)
17     @test abs(g-jacobian) < 6*s
18 end

```

For each of three particle counts $J \in \{200, 1000, 5000\}$, the classic bootstrap filter (`trigger = 1`, `target = 0`, the package defaults) is run 30 times and the 30 log-likelihood replicates combined via `logmeanexp` (a numerically stable log of an average of exponentials, with a jackknife standard error) into a single estimate `x.est` with reported Monte Carlo standard error `x.se` at that J . `gap` is the difference between this estimate and the exact log-Gompertz-scale Kalman likelihood `ll_logy`, which by the Jacobian identity above should equal `jacobian` up to Monte Carlo error at every J tested — *not* shrink toward zero as J grows.

Primary check. A genuine defect in the filter or in the model definition — as opposed to a mere additive bookkeeping error such as an incorrectly signed Jacobian — would generically make `gap` drift *systematically with J* : the textbook downward bias of the log of a Monte Carlo average of a nonnegative unbiased estimator scales as $-\text{Var}/(2J)$ (a Jensen’s-inequality effect on $\log \hat{\mathcal{L}}$, not on $\hat{\mathcal{L}}$ itself) and shrinks toward zero as $J \rightarrow \infty$; a fixed bookkeeping constant, by contrast, shifts `gap` by the same amount at every J . The primary check tests exactly this distinction: $\max(\text{gap}) - \min(\text{gap})$, the spread of `gap` across the three particle counts, must be small relative to `tol_spread = 6 × the largest (noisiest, smallest- $J$ ) standard error`. This test is run only at the classic default settings (`trigger = 1`, `target = 0`), where Proposition 1(a) already guarantees that the resampling bookkeeping question examined in Section 3 — whether a factor is credited back at resampling, or left to ride into the next step’s baseline — cannot make any difference: the factor mS'/J in question is identically one at $\beta = 0$. This check therefore neither tests, nor could have tested, that question one way or the other; Section 3 settles it instead with a dedicated, paired numerical comparison at intermediate β , run directly against this same exact reference likelihood (Section 3.5).

Secondary check. `gap` should also be close, in absolute terms, to `jacobian` itself, at every J : this confirms that no fixed bookkeeping error of the kind the primary check is designed to be

insensitive to is nonetheless actually present — i.e., that the two checks together pin down both ”no systematic drift with J ” and ”the correct fixed constant,” leaving no room for an error that happens to be constant in J but wrong in value.

8 History: a credit added and retracted

This document’s account of resampling bookkeeping did not arrive at Section 3 in one pass, and it is worth recording the false step along the way, since what it does and does not show is itself instructive.

Two genuine defects were found while writing this document, both by the same method: mapping each routine in `pfilter.jl` onto its mathematical specification, line by line, and checking that the two actually agreed. Section 2 found that an earlier translation of the tempered resampling step had indexed the retained weight by selection *slot* rather than by selected *ancestor* — assigning particle k the quantity $(w^k)^\beta$ instead of $(w^{p_k})^\beta$, a defect invisible at $\beta = 0$ (where the retained-weight power is trivial) but which targets the wrong measure at every $\beta > 0$. Section 6 found the same error’s cousin in the terminal ancestry draw: a uniform draw over terminal particle *indices* where the terminal particles are not, in general, equally weighted. Both were fixed in the code.

A third, superficially similar candidate defect was proposed and did not survive scrutiny. While deriving the telescoping identity term by term for the tempered resampling step (Section 3), it appeared that the identity $\hat{\mathcal{L}}(\theta) = \prod_n \hat{\mathcal{L}}_n$ could only be maintained at intermediate $\beta \in (0, 1)$ if the factor mS'/J removed by unit-mean renormalization at resampling — the quantity of Proposition 1 — were explicitly credited back into the running log likelihood. This looked, at the time, like a defect of exactly the same kind as the two above: real, invisible at the two settings ($\beta = 0$, $\beta = 1$) that earlier validation work (against the Kalman filter, Section 7, and against R `pomp`, the memo in this directory) had exercised, and fixable by a one-line change. The change was made — `systematic_resample!` was altered to *return* the log credit $\log(mS'/J)$ in place of `nothing`, and `pfilt_step_comps!` was altered to add that returned value into `logLik[]` — and it stayed in the code for a time.

The error was in the inference, not the algebra: mS'/J is exactly the factor by which unit-mean renormalization departs from the properly weighted target of Section 3.2, and crediting it back does restore that particular sufficient condition for unbiasedness. What does not follow is that failing to credit it leaves the estimator biased. Proper weighting is one route to an unbiased telescoping product; it is not the only one. The route the uncredited code actually takes — Section 3.4’s observation that a mean-one factor absorbed silently into the next step’s baseline costs only variance, never expectation, exactly as Aaron King’s R `pomp` implementation has always assumed without remark — was overlooked at first, because the draft reasoned directly from ”this sufficient condition fails” to ”the estimator is biased,” without checking whether a different sufficient condition might hold instead.

What exposed the mistake was the numerical comparison of Section 3.5: the same Gompertz–Kalman reduction used for the general validation of Section 7, but run as a paired comparison of the credited and uncredited running log likelihoods computed from identical particle draws, at intermediate β specifically — exactly the regime the original argument said should show a detectable, systematic difference. It did not: across $\beta \in \{0.25, 0.5, 0.75\}$ and $J \in \{8, 16, 64\}$, with 400,000 paired replicates each, the credited and uncredited estimators agreed to within 0.003% of $\hat{\mathcal{L}}$ everywhere, with no z -score approaching significance. At that point the credit was removed from the code again, restoring the bookkeeping described in Section 3 (no credit computed, no credit

returned, `systematic_resample!` once more returning `nothing`), and this document’s account of the resampling step was rewritten to explain why that bookkeeping was right all along.

It is worth being explicit about what the two validation tools used in this document each can and cannot see, since the credit episode is a clean illustration of the gap. The Gompertz–Kalman comparison of Section 7, run at the classic default settings, checks the *marginal likelihood* — a single scalar, averaged over the randomness of the particle swarm — against an exact reference. Neither of the two genuine defects disturbs that quantity: mislabeling which ancestor’s retained weight lands on which selected particle changes which states carry which weights, and the uniform terminal draw changes which trajectory gets reported at the end, but neither changes the *expected value of the sum of weights* at any observation time, which is all the marginal likelihood ever sees. A comparison of average log likelihoods, however extensive, could not have caught either one; they were found only by reading the code against its mathematical specification, line by line, as Sections 2 and 6 do. The credit question is the opposite case: it is specifically and only a claim about the running log-likelihood accumulator, and a paired comparison of that accumulator against an exact reference is exactly the right instrument for it — which is why the numerical check of Section 3.5 that found nothing is good evidence there was nothing to find, in a way that the (correctly) clean result of Section 7’s classic-default check could never have been for the credit question, since that check runs at $\beta = 0$, where Proposition 1(a) guarantees the credited and uncredited schemes coincide regardless of which is right.

The lesson is not that the derivation in Section 3 was wasted effort, or that careful term-by-term mathematical accounting is not worth doing — the same method caught two real defects. It is narrower and, perhaps, more useful: proper weighting is a *sufficient* condition for an unbiased sequential Monte Carlo likelihood estimator, not a *necessary* one, and a resampling scheme can fail it while still being exactly unbiased by a different route. Concluding bias from the failure of one sufficient condition, without checking whether another one applies, is the specific mistake this document made and then corrected.